

Trabalho 1 - Análise de complexidade em estruturas de listas.

Regras Gerais

- Os trabalhos devem ser desenvolvidos em grupos de **até 3 pessoas**. O ideal é que todos os trabalhos futuros sejam desenvolvidos neste mesmo grupo.
- Os grupos devem se registrar [na planilha disponibilizada no Ava](#) para esse fim.
- Os relatórios devem ser entregues em formato PDF através do AVA.
- Apenas um componente de cada grupo deve enviar o relatório, onde deve constar os nomes de todos os componentes do grupo.
- O prazo para entrega do relatório está definido na respectiva atividade no Ava. Não serão aceitas entregas atrasadas.
- Todos os códigos desenvolvidos, bem como o relatório, devem ser disponibilizados em um repositório do github, cujo endereço deve estar especificado no relatório.
- Para a definição da nota serão considerados os códigos desenvolvidos, o relatório entregue e as entrevistas.
- Nas entrevistas os alunos deverão apresentar os códigos desenvolvidos e responder a perguntas referentes às análises apresentadas no relatório. As perguntas podem ser direcionadas a um aluno específico ou ao grupo.
- Alunos de um mesmo grupo podem ter notas diferentes, dependendo da participação nas atividades e do desempenho na entrevista. A falta de participação do aluno nas atividades, bem como o desempenho muito ruim na entrevista pode levar até mesmo a zerar a nota do trabalho.
- Este trabalho valerá 25 pontos (25% da nota do semestre).

Introdução

O objetivo deste trabalho é revisar conceitos de estruturas de dados e de programação orientada a objetos em Java, além de aplicar conceitos de análise de complexidade de algoritmos para avaliar os códigos de lista gerados e realizar alguns testes empíricos para verificar a complexidade dos códigos gerados.

Neste trabalho vamos: (i) implementar nossa própria biblioteca de lista genérica em Java; (ii) analisar matematicamente a ordem de complexidade dos métodos da nossa biblioteca; (iii) fazer testes empíricos para avaliar o tempo de execução dos métodos da nossa biblioteca e (iv) refletir sobre os tempos obtidos na execução e a ordem de complexidade dos métodos.

Neste trabalho serão produzidos códigos em Java e um relatório seguindo as especificações a seguir.

Especificações do Trabalho e do Relatório a ser entregue

1) Prática de Implementação: Implementação de uma biblioteca de Listas em Java usando Generics

Você deve implementar uma biblioteca de lista encadeada em Java que permita a manipulação de listas genéricas utilizando *Generics*. A sua biblioteca deve atender aos seguintes requisitos:

- A classe que representa a lista deve se chamar “ListaEncadeada”;
- Os nós da lista devem armazenar valores de um tipo genérico “T”;
- A classe “ListaEncadeada” deve ter um método construtor que receba como parâmetros: (i) uma instância de um `Comparator<T>`, que será utilizado para comparar os elementos da lista sempre que necessário; (ii) um boolean que indicará se a lista deve ser ordenada (true) ou não-ordenada (false)
- A classe “ListaEncadeada” deve sobrescrever o método `toString` para devolver uma String iniciada por “[” e finalizada por “]” concatenando no meio os `toString` de todos os valores armazenados na Lista, separados por vírgula (“,”);
- A classe “ListaEncadeada” deve implementar a interface “`IColecao`” disponibilizada no [github](#);

Dica: Quase todo o código necessário está disponível nos slides de revisão e [no repositório do github](#) há um exemplo para estruturação do código!!!

2) Prática de Implementação: Implementação de um programa que use a biblioteca

Para testar sua biblioteca e exemplificar o uso da mesma crie um programa que seja capaz de armazenar dados de contatos (e todas as classes de domínio necessárias). Cada contato terá um nome e um telefone.

Seu programa deve atender aos seguintes requisitos:

- Ao iniciar seu programa deve perguntar ao usuário se deseja uma lista ordenada ou não ordenada e instanciar a(s) lista(s) do tipo escolhido usando a biblioteca feita no passo anterior. A(s) lista(s) instanciada(s) deve(m) ser armazenada(s) em variável(is) do tipo `IColecao`;
- Então o programa deve exibir um menu com as seguintes opções:
 - Carregar dados de arquivo: nessa opção o programa deve ler um arquivo de nome “entrada.txt” que conterá dados de vários contatos e os inserir na(s) lista(s). O formato do arquivo fica a critério do grupo. Ao acabar de ler o arquivo e construir as listas, o programa deve exibir o tempo total gasto para leitura do arquivo e montagem das listas.
 - Adicionar contato: o programa deve solicitar o nome e o telefone do contato e adicioná-lo à(s) lista(s). Não deve ser permitido dois contatos com o mesmo telefone (isso é uma regra de negócio e não da sua lista, ou seja, essa regra não deve afetar em nada o código da sua lista).
 - Pesquisar contato por nome: o programa solicita o nome do contato e exibe seu telefone ou uma mensagem falando que ele não existe. Depois exibe

também o tempo gasto na execução do método de busca. Caso exista mais de um contato com o nome dado, será exibido apenas um deles;

- Pesquisar contato por telefone: o programa solicita o telefone do contato e exibe seu nome ou uma mensagem falando que ele não existe. Depois exibe também o tempo gasto na execução do método de busca;
- Remover contato por telefone: o programa deve solicitar o telefone do contato e excluí-lo da(s) lista(s). Em seguida, exibe uma mensagem informando que o contato foi excluído (ou que ele não existia) e o tempo gasto na execução do método de remoção.
- Alterar dados de contato: o programa solicita o nome do contato e exibe seu telefone. Em seguida, pergunta novo nome e telefone, alterando os dados do contato pelos novos dados.
- Sair: exibe a quantidade total atual de contatos e o programa é encerrado.

Requisitos de modularização:

- Deve ser criada uma classe de domínio “Contato” que tenha os atributos nome e telefone e que sobrescreva o método toString para que ele devolva o nome e o telefone do contato separados por um hífen (“-”).
- **Nenhuma mensagem de interação com o usuário deve ser impressa pelos métodos da biblioteca!**

Outras regras:

- Se você utilizar duas listas, de forma a usar uma para buscas por nome e outra para busca por telefone, os contatos não podem ser duplicados!
- Todo o código desenvolvido deve estar disponível em um repositório do github.

3) Primeira seção do relatório: relato sobre o desenvolvimento da biblioteca e do programa de teste

A entrega deste trabalho se dará por meio de um relatório. A seção 1 deste relatório deve:

- Descrever a atuação de cada um dos componentes do grupo no desenvolvimento dos itens 1 e 2 desta especificação;
- Descrever, detalhadamente, como ferramentas de IA foram utilizadas para o desenvolvimento do trabalho.
- Indicar o endereço do repositório do github onde o projeto estará disponível. O repositório deve ter um README descrevendo a organização do código e como rodá-lo.

4) Segunda seção do relatório: Análise Matemática de complexidade dos Algoritmos da sua biblioteca

Na seção 2 do relatório você deve desenvolver análises de complexidade dos métodos adicionar, pesquisar, remover e quantidadeNos da sua biblioteca. As análises devem referenciar os códigos implementados, explicando o raciocínio linha a linha. Assim, é recomendável que se use no relatório imagens dos códigos com as linhas numeradas para facilitar a referência. Para cada método analisado você deve chegar a conclusão sobre qual a ordem de complexidade no pior caso e explicar quando esse pior caso ocorre. Você deve discutir se, pelo seu código, há diferenças na ordem de complexidade entre listas ordenadas e listas não-ordenadas.

5) *Análise Empírica de complexidade de Algoritmos da sua biblioteca*

Para analisar empiricamente a complexidade dos algoritmos de sua biblioteca, você deve rodar diversas vezes seu programa. Cada vez que rodar seu programa você deve dar como entrada um arquivo grande, de diferentes tamanhos (por exemplo, 100000, 200000, 400000 elementos). Para gerar os arquivos de entrada você pode utilizar uma LLM ou construir um programa que gere arquivos com dados aleatórios. Exemplos de programas geradores de arquivos estão disponíveis em <https://github.com/victoriocarvalho/GeradorArquivos>.

Para cada tamanho de arquivo (ao menos 4 tamanhos), rode escolhendo lista não-ordenada e depois lista ordenada. A cada execução você deve:

- Anotar o tempo gasto para leitura do arquivo e montagem da lista;
- Pesquise pelo telefone do último elemento da lista e anote o tempo gasto na execução do método de busca;
- Pesquise pelo nome do último elemento da lista e anote o tempo gasto na execução do método de busca;
- Remova o último contato da lista por telefone e anote o tempo gasto na execução do método de remoção;

Então, na seção 3 do relatório você deve:

- Apresentar tabelas e/ou gráficos exibindo os tempos coletados em função do tamanho do arquivo de entrada. Faça uma tabela/gráfico para cada operação que teve o tempo medido. Assim você terá (i) uma tabela/gráfico que mostre o tempo de montagem das listas em função do tamanho da entrada; (ii) uma tabela/gráfico que mostre o tempo de pesquisa pelo último contato em função do tamanho da entrada; (iii) uma tabela/gráfico que mostre o tempo de remoção do último contato em função do tamanho da entrada;
- Interprete os dados e tempo coletados, discutindo se eles atendem ao esperado considerando as análises matemáticas feitas na seção anterior.